

One monthly preventive maintenance from birth → execution → manager dashboard,
using the tables we designed.

Contents

Phase 0 – One-time setup (done once per type of PM).....	2
0.1. Define the PM template (by Admin / Engineer)	2
0.2. Attach the plan to a restaurant/equipment.....	2
Phase 1 – When the monthly PM “comes due”	3
1.1. Scheduler finds due assignments (daily/cron job)	3
Phase 2 – Technician executes the PM visit	3
2.1. Technician sees their assigned PMs	3
2.2. When technician opens the work order (first time).....	3
2.3. Technician records the maintenance results.....	4
2.4. System (or tech) creates Non-conformities if needed.....	4
2.5. Technician completes the work order	4
Phase 3 – Manager of the restaurant visualizes everything	5
3.1. Manager logs in and sees “Monthly PM status”	5
3.2. Optional – PM report PDF for authorities	6

Phase 0 – One-time setup (done once per type of PM)

0.1. Define the PM template (by Admin / Engineer)

Goal: Create a reusable “Monthly Preventive Maintenance for Extraction” (for example).

Tables involved:

- maintenance_plan_templates
- maintenance_plan_tasks

Process:

1. Admin creates a template:
 - Insert into maintenance_plan_templates:
 - name = 'Monthly Extraction PM'
 - description = 'Monthly checklist for extraction hoods & fans'
 - category_id → e.g. EXTRACTION
 - applies_to = 'RESTAURANT' (or 'EQUIPMENT' if per fan/hood)
 - frequency_type = 'MONTHLY'
 - frequency_days = NULL (only used for custom)
 - default_duration_min = 90
 - is_active = 1
 - created_by_user_id = admin user
2. Admin defines checklist items:
 - Insert rows in maintenance_plan_tasks linked by plan_id:
 - Task 1: section = 'Extraction & Ventilation', description = 'Check and clean hood filters', expected_result_type = 'BOOLEAN'.
 - Task 2: section = 'Extraction & Ventilation', description = 'Measure fan motor current', expected_result_type = 'NUMERIC', unit = 'A', min_value, max_value.
 - Task 3: ... etc.

At this point, we have a **generic PM plan** defined, but not yet assigned to any restaurant.

0.2. Attach the plan to a restaurant/equipment

Goal: Say Bam Bam Glòries wants this monthly PM on their extraction system.

Tables:

- maintenance_assignments
- (linking to restaurants, optionally restaurant_sections or equipment)

Process:

1. For restaurant “Bam Bam Glòries”, admin creates a maintenance assignment:
 - Insert into maintenance_assignments:
 - plan_id = ID of “Monthly Extraction PM”
 - restaurant_id = Bam Bam Glòries
 - section_id = (optionally “Kitchen” section)
 - equipment_id = NULL (if the plan is for the whole extraction system), or a specific fan.
 - active = 1
 - start_date = '2025-01-01'
 - last_completed_date = NULL (first time)
 - next_due_date = '2025-02-01' (for example)
 - notes = 'Monthly extraction PM as per contract'

Now the system knows: **every month**, for this restaurant (and section/equipment), a PM visit is required.

Phase 1 – When the monthly PM “comes due”

1.1. Scheduler finds due assignments (daily/cron job)

Goal: On/after the due date, create a **work order** for the technician.

Tables:

- maintenance_assignments
- work_orders

Process (e.g. daily at 02:00):

1. Cron job runs a query like:
 - Select all rows from maintenance_assignments where:
 - active = 1
 - next_due_date <= TODAY
 - (and no open WO already created for that due date, if you track that)
2. For each assignment found, create a **PREVENTIVE work_order**:
 - Insert into work_orders:
 - restaurant_id = assignment.restaurant_id
 - section_id = assignment.section_id
 - equipment_id = assignment.equipment_id
 - assignment_id = assignment.id
 - type = 'PREVENTIVE'
 - status = 'OPEN'
 - priority = 'MEDIUM' (or based on criticality)
 - summary = 'Monthly Extraction Preventive Maintenance'
 - description = plan.description or generated text
 - created_by_user_id = system/admin
 - assigned_to_user_id = default technician for that restaurant (or null for dispatcher to assign)
 - scheduled_start = date/time of planned visit (could be same as next_due_date)

Now there is a **work order** in the system waiting for a technician.

Phase 2 – Technician executes the PM visit

2.1. Technician sees their assigned PMs

Views:

- Tech mobile app calls API to list work_orders:
 - Filter: assigned_to_user_id = current_user
AND type = 'PREVENTIVE'
AND status IN ('OPEN','IN_PROGRESS').

They see “Monthly Extraction Preventive Maintenance – Bam Bam Glòries”.

2.2. When technician opens the work order (first time)

Goal: Materialize the checklist for this specific visit.

Tables:

- maintenance_plan_tasks
- work_orders
- work_order_tasks

Process:

1. App loads **work_order** and its associated assignment:
 - assignment_id → plan_id (“Monthly Extraction PM”).
2. Query maintenance_plan_tasks for that plan_id.
3. For each template task, create a corresponding **work_order_tasks** row (if choose to instantiate them on first open):
 - Insert into work_order_tasks:
 - work_order_id = current WO id
 - plan_task_id = template task id
 - section = template.section
 - task_order = template.task_order
 - description = template.description
 - unit, etc.
 - result_* = NULL initially
4. Optionally, set WO status to 'IN_PROGRESS' and set started_at = now() when tech starts.

The technician now has a **checklist UI** tied to work_order_tasks.

2.3. Technician records the maintenance results

During the visit, for each task:

- The app sends updates to work_order_tasks:
 - For boolean tasks:
 - result_boolean = 1 or 0
 - For numeric tasks:
 - result_numeric = e.g. 4.8, unit = 'A'
 - For textual notes:
 - result_text = 'Slight vibration noise, within acceptable'

If a task reveals a problem:

- Mark is_non_conformity = 1 on that work_order_tasks row.

Technician can also:

- Add **photos** linked to work_order_id (and equipment_id if needed).

2.4. System (or tech) creates Non-conformities if needed

Tables:

- non_conformities

For each task with is_non_conformity = 1 (or values out of allowed range), it creates:

- Insert into non_conformities:
 - work_order_id = current WO
 - work_order_task_id = that task id
 - restaurant_id = WO.restaurant_id
 - equipment_id = WO.equipment_id (or null if general)
 - severity = 'HIGH' / 'MEDIUM' etc.
 - description = “Fan motor overcurrent (5.6 A > 4.0 A threshold).”
 - recommendation = “Plan a corrective intervention to inspect motor bearings.”
 - status = 'OPEN'
 - created_at = now()

Now each issue is traceable across visits for authorities and the manager.

2.5. Technician completes the work order

When all tasks are done:

- Update **work_orders**:

- status = 'COMPLETED'
- completed_at = now()

Then update **maintenance_assignments**:

- last_completed_date = TODAY
- Compute next_due_date:
 - For MONTHLY: next_due_date = last_completed_date + interval of ~1 month
 - Store as 'YYYY-MM-DD' in next_due_date.

Now the system is ready to schedule the **next month's PM**.

Phase 3 – Manager of the restaurant visualizes everything

3.1. Manager logs in and sees “Monthly PM status”

The manager is a users row with:

- Entry in restaurant_user_access for their restaurant (role_at_site 'MANAGER' or 'OWNER').

Views & queries:

1. Dashboard – Upcoming maintenance

- Query maintenance_assignments where:
 - restaurant_id = manager_restaurant_id
 - active = 1
 - order by next_due_date.
- Show a list like:

Plan name	Applies to	Next due date
Monthly Extraction PM	Kitchen	2025-03-01
Monthly Refrigeration PM	Entire	2025-03-05

2. History – Completed PM visits

- Query work_orders:
 - restaurant_id = manager_restaurant_id
 - type = 'PREVENTIVE'
 - status = 'COMPLETED'
 - Filter by date range (e.g. last 6 months).

Show for each:

- Date (completed_at)
- Type (“Monthly Extraction PM”)
- Technician name
- Any non-conformities count.

3. Drill-down – Visit details

When manager clicks a specific WO:

- Load work_orders row.
- Load related work_order_tasks:
 - Show checklist with results (OK / Not OK / measures).
- Load related photos (before/after pictures).
- Load related non_conformities (if any):
 - Show severity, description, recommendation, and status.

4. Non-conformities / Issues view

Separate tab showing:

- Query non_conformities where:
 - restaurant_id = manager_restaurant_id
 - status IN ('OPEN','IN_PROGRESS')

Show:

Date	Equipment	Severity	Description	WO ref
2025-02-01	Hood Fan #1	HIGH	“Motor overcurrent ...”	WO-123
2025-02-01	Extraction Duct	MEDIUM	“Grease accumulation over limit”	WO-123

This gives the manager **full visibility**: what was done, when, by whom, and what problems remain.

3.2. Optional – PM report PDF for authorities

Once we have all this in DB, generating a PDF for inspections is straightforward:

- Use data from:
 - work_orders, work_order_tasks, non_conformities, photos, documents.
- Build a report: “Monthly Extraction PM – February 2025”, with checklist and signatures.